

Intelligent Disaster Monitoring and Emergency Communication Network

Initial report — what the project is, why it is needed, why it is built entirely in software, and how it works.

1. What This Project Is

This is a web platform that watches for disasters, decides how serious they are, warns exactly the people who are in danger, and then coordinates the rescue response — and it keeps working even when the mobile network fails.

It has two halves, matching the two halves of the title. **Monitoring** means continuously pulling hazard data from public sources (earthquakes, rainfall, satellite-detected fires, global alert feeds) and combining it with emergency reports sent in by ordinary people from their phones. **Emergency communication** means turning that into a targeted warning delivered to specific individuals, and then running the response — verifying incidents, dispatching teams, routing them around blocked roads, and recording every decision.

The project is **entirely software**. There is no sensor, no microcontroller, no physical device built anywhere in it. Section 5 explains why that is the right engineering choice and not a shortcut.

2. The Problem

The instinctive assumption is that people die in disasters because nobody saw the disaster coming. That is no longer true. Weather services, seismic networks and Earth-observation satellites detect hazards reliably, and they publish that data openly and free of charge.

Disasters are not missed. Warnings are. The bottleneck is no longer detection — it is communication, verification and coordination. That entire gap is software.

The same failures repeat in event after event:

- **Fragmented data.** Seismic, weather and satellite information sit in separate portals in different formats. Nobody has one operational picture; officials check several websites by hand.
- **Untargeted warnings.** Alerts are issued for a whole district or state. Someone 60 km from a flood gets the same message as someone 600 m from it, so neither can judge whether they personally need to move.
- **No way for citizens to report in.** People on the ground see the danger first — a cracking embankment, a blocked drain, a collapsed wall — but the only channel is a voice helpline. That ground truth is lost.
- **Duplicate chaos.** One flood generates hundreds of separate calls. The control room cannot tell whether it is dealing with one large event or twenty small ones.
- **Manual coordination.** Teams are dispatched by phone and tracked on paper or in chat groups. Nobody has a live picture of who is where.
- **The network fails.** Towers lose power and get congested exactly when the disaster peaks — so any purely online system dies at the moment it matters most.
- **No feedback and no record.** Authorities cannot tell whether a warning was actually delivered, whether a person is safe, or afterwards, who authorised what.

There is a second-order consequence that is easy to miss. A system that sends irrelevant warnings teaches its users to ignore warnings. After enough false or untargeted alerts, people stop reading them — and the system then fails *silently*, appearing to work while actually reducing how fast people react. **Over-alerting is worse than not alerting.**

3. What Already Exists, and What It Leaves Out

Plenty of systems solve part of this. None solves all of it.

System	What it does well	What it leaves out
Weather and seismic agencies	Authoritative, calibrated, scientific detection	Publishes data and bulletins, not personalised warnings. No citizen input. No response coordination.
Mass SMS / cell broadcast	Reaches very large populations quickly	One-way only. Coarse targeting. No verification, no acknowledgement, no coordination layer.
Ushahidi (crowdmapping)	Excellent citizen reporting and mapping	No automated hazard-feed ingestion, no severity engine, no geofenced alert dispatch.
Sahana Eden	Strong shelter and resource management	Heavyweight, not real-time-first, weak on automated feed ingestion and mobile/offline use.
Social media	Fast, ubiquitous, high participation	Unstructured, unverified, rumour-prone, not geographically queryable, no official workflow.

The unoccupied space is a system that does all of the following at once: ingests several authoritative feeds automatically, fuses them with structured citizen reports, merges duplicates into one incident, scores severity transparently, requires a human to verify before alerting, targets alerts by actual location, tracks delivery per person, coordinates the response, survives network loss, keeps an unchangeable audit trail, and costs nothing to run. That is what this project builds.

4. What the System Must Actually Do

Before choosing how to build it, it is worth being precise about what it has to achieve. Eight things:

1. Pull hazard data from at least four independent public sources, continuously and without anyone pressing a button.
2. Merge related signals and reports into a single incident with a confidence score, instead of producing one alarm per report.
3. Work out who is physically inside an affected area, fast enough to be useful during an emergency.
4. Deliver alerts across several channels, retry the ones that fail, escalate when nobody responds, and record what reached whom.
5. Accept emergency reports from citizens even when there is no network connection at all.
6. Give a control room a live picture and the tools to verify incidents, dispatch teams and route them around blocked roads.
7. Make sure each role can only do what that role is allowed to do, and write every significant action to a record that cannot be altered afterwards.
8. Run publicly, at zero recurring cost, and prove itself against measurable targets rather than screenshots.

5. Two Ways to Build It — and Why Software Wins

Two approaches were designed in full before choosing. The obvious one for a disaster-monitoring project is a hardware sensor network. It was evaluated seriously and rejected for reasons that are worth setting out properly.

Option A — a physical sensor network. Raspberry Pi or ESP32 nodes carrying an ultrasonic water-level sensor, a rain gauge, a vibration sensor, a smoke sensor and a GPS module, powered by solar and battery, in a weatherproof enclosure, reporting over LoRa or GSM to a cloud server. Realistic cost: **Rs. 6,000 to Rs. 15,000 per node.**

Its genuine strengths, stated fairly:

- It produces original physical measurements no public feed provides.
- It is physically demonstrable — an examiner can watch a device react.
- It gives hyper-local resolution no satellite can match.
- With LoRa it can work independently of internet infrastructure.

Why it fails as a solution to this problem:

1. **Coverage — the fatal one.** One sensor monitors roughly one point. A flood covers hundreds of square kilometres. Monitoring even one medium city at useful density needs *thousands* of nodes; a student project can build one to three. You could prove that a sensor works, but you could never demonstrate the actual thing being claimed — a network monitoring a region.
2. **Cost does not scale.** At Rs. 6,000 to 15,000 per node, real coverage runs into crores. There is no path from the prototype to anything deployable.
3. **Deployment is legally impossible.** Installing sensors on rivers, bridges, embankments or public infrastructure needs permissions from municipal, irrigation and disaster authorities that no student obtains in a semester. In practice the demo becomes a sensor in a bucket of water on a lab bench, which evidences nothing.
4. **Power and connectivity fail exactly when needed.** Disasters cause outages and tower failures. The sensor most likely to go offline is the one inside the disaster.
5. **Calibration is out of reach.** Hobby-grade sensors drift and are temperature sensitive. An uncalibrated water-level reading cannot be the basis of a public evacuation order. Establishing that credibility is a metrology problem, not a semester project.
6. **Maintenance never ends.** Weatherproofing, corrosion, debris and spider webs blocking ultrasonic sensors, battery replacement, theft and vandalism.
7. **It duplicates national infrastructure, worse.** Meteorological radar networks, national seismic networks and Earth-observation satellites already exist, are rigorously calibrated, and publish their data free. Three uncalibrated hobby sensors add no information those systems do not already provide more accurately over a vastly larger area.
8. **Demonstration risk.** A dry solder joint, a flat battery, no Wi-Fi, interference in the exam hall. It cannot be re-run reliably on demand from a laptop.
9. **Wrong discipline.** Circuit design, soldering, power management and enclosure fabrication are Electronics work. In a Computer Science project they displace exactly the things the project should be demonstrating.
10. **It solves the wrong problem.** Detection is not the bottleneck. Adding a fourth-rate detector to a world that already has first-rate detectors saves nobody. People are lost in the gap between detection and action — and that gap is made of communication, verification, targeting and coordination.

Option B — a software platform. Instead of competing with the world's sensing infrastructure, consume it. The platform pulls from satellites, seismic networks and weather models that are already calibrated and already free, then adds citizen smartphones on top, then builds the missing layer.

	Hardware sensor network	Software platform
Coverage	1 to 3 points	Worldwide from day one
Cost	Rs. 6,000 to 15,000 per node	Rs. 0 total
Deployment	Permissions unobtainable	A public URL, live today
Sensor quality	Uncalibrated hobby components	Agency-grade calibrated data
Extra sensors	Must be built and installed	Every user's smartphone, free
During a disaster	Loses power at the worst moment	Offline-first app keeps working
Problem addressed	Detection, already solved	The last mile, not solved
Discipline	Mostly Electronics	Core Computer Science

Scored across fifteen weighted criteria — coverage, cost, scalability, deployability, whether it addresses the stated problem, demonstration reliability, data credibility, maintenance, curriculum fit, resilience, users servable, skill fit, extensibility, originality of measurement, and time to a working system — the result was **hardware 37 percent, software 93 percent**. The only criterion hardware wins is originality of physical measurement, which carries the lowest weight, because original measurement is not the problem this project exists to solve.

6. If We Build No Sensors, What Is Doing the Sensing?

This is the fair objection to a purely software system, and it has a clean answer. There are two sensing layers, both of which already exist and neither of which needs to be built.

Layer 1 — institutional instrumentation, the wide layer. Earth-observation satellites, national and global seismic networks, Doppler radar, automatic weather stations and numerical weather models. Decades of investment, rigorous calibration, continuous global coverage — and the output is published openly for free. The project effectively operates a research-grade sensor network at zero cost.

Layer 2 — citizen smartphones, the deep layer. Every person who installs the app contributes a device with GPS, a camera, a network radio, a battery and a screen — and, uniquely, a human being attached to it who can interpret what they are seeing. This supplies exactly what satellites cannot: the detail of one street, one building, one embankment.

No physical instrument, at any price, can report: "The embankment on the east side has cracked, water is coming through, and about forty families are still inside."

Global coverage from one layer, street-level resolution from the other, with nothing to manufacture, install, power, calibrate or maintain.

7. How the System Works

The platform is eight components in one pipeline:

#	Component	What it does
1	Ingestion worker	Polls each public data source on its own schedule, converts wildly different formats into one common shape, and discards anything already seen.
2	Incident engine	Decides whether a new signal belongs to an existing incident or starts a new one, scores its severity, and enforces the incident lifecycle.
3	Geofence matcher	Answers the central question: exactly who is inside this affected area right now?
4	Alert dispatcher	Fans the alert out across channels with queueing, retries, escalation and per-person delivery tracking.
5	Citizen app	SOS reporting, offline queue, live map, shelter finder, "I'm safe" check-in.
6	Responder console	Assigned tasks, incident detail with photos, routing that avoids blocked roads, status updates.
7	Admin and analytics	User management, data-source health, response-time metrics, unchangeable audit log.
8	Realtime gateway	Pushes every change live to connected dashboards, so nobody refreshes a page.

Scenario 1 — a hazard detected from official data

09:04 — The weather source reports 118 mm of rain in the last 24 hours over Zone 4. The ingestion worker records it. No human is involved.

09:04 — The rules run: rainfall is above the flood threshold, and there is no matching open incident nearby, so a new incident is created. Severity is computed from rainfall intensity, forecast trend, population density and time of day: **78 out of 100, tier WARNING**.

09:04 — It appears instantly at the top of the coordinator's dashboard, in red. Nobody refreshed the page; the live connection pushed it.

09:06 — The coordinator reviews the underlying data and clicks **Verify**. Only now is the system allowed to alert the public. This single human step is what stops automated false alarms from ever reaching people.

09:07 — The coordinator draws the alert zone directly on the map. Before sending, the screen shows: *"this will reach approximately 1,240 people"*.

09:07 — 1,240 phones are alerted. Someone 40 km away receives nothing. That is the difference between a broadcast and a warning.

09:08 — The dashboard shows live delivery: **delivered 1,198, pending 25, failed 17**. For the first time the control room knows whether the warning actually arrived.

Scenario 2 — a citizen sees it first

09:15 — A resident watches a wall collapse onto a road. He opens the app, presses the red SOS button, picks "structure collapse", the app grabs his GPS automatically, he takes a photo and adds one line. About eight seconds.

09:15 — A new incident is created but marked **unverified**, because a single report from one stranger is not evidence enough to alert a population.

09:17 — Two other people report the same thing independently. The system recognises all three as the same event, merges them into **one** incident, and raises the confidence.

09:18 — With corroboration it climbs the priority queue. The coordinator opens it, looks at the three photographs, verifies it, and assigns a rescue team.

In a real disaster one flood may produce five hundred reports. The control room must see **one incident supported by five hundred reports** — not five hundred incidents. Without this merging step, situational awareness collapses at exactly the moment it is needed most.

Scenario 3 — when the network fails

09:32 — A woman is trapped on the ground floor of a flooded building. Her mobile data is dead: the local tower has lost power and the network is congested.

She opens the app anyway — it is already stored on her phone and works with no connection. She presses SOS and submits. The screen confirms: *"Report saved. It will be sent automatically as soon as connectivity returns."*

09:51 — She moves upstairs and catches one bar of signal. Without her opening the app, without her doing anything at all, the phone transmits the queued report by itself.

09:51 — It lands on the coordinator's dashboard stamped with the **original** time of 09:32, not the time it was transmitted — so the control room knows how long she has been waiting.

This is demonstrated simply: turn on aeroplane mode, submit a report, turn it off, and watch it appear on the control-room screen.

Scenario 4 — the response, and afterwards

The assigned team receives a task card with the hazard, severity, distance and a deadline. They tap it and get a map with a route drawn to the location — a route that avoids the roads the coordinator has already marked as flooded, which no ordinary navigation app knows about. As they work they tap through statuses: on the way, arrived, in progress, completed. The coordinator sees every step live. Nobody phones anyone to ask where the team is.

When the water recedes the incident is marked resolved. The system now holds a full record: when it was detected and from which source, how many people reported it, how long verification took, how many were alerted on which channels, how many marked themselves safe, which team responded and how long it took. Every verification, alert and assignment sits in a log that cannot be edited or deleted — which in real disaster management matters as much as the rescue itself.

8. What the Portal Contains

One application, one login, four different interfaces depending on who you are. Roughly 33 screens built on about 30 shared components.

Role	What they get
Citizen	Big SOS button, live incident map, alerts inbox, nearest shelters with live occupancy, "I'm safe" check-in, my-reports history, alert preferences. Works offline.
Responder	Priority-sorted task list with deadlines, incident detail with citizen photos, navigation with blocked roads avoided, status buttons, field report form.
Coordinator	Three-panel command dashboard - live incident queue, map, and action panel. Verify, reject, draw an alert zone, compose and send, watch delivery counts, assign teams, manage resources and shelters, mark roads blocked.
Administrator	User and role management, data-source health cards, analytics on response and delivery times, unchangeable audit log, editable hazard thresholds, scenario simulator.

The coordinator's dashboard is the centrepiece: incident queue on the left sorted by severity, live map in the middle showing incidents, responder positions, shelters and blocked roads, and the selected incident's detail and actions on the right. Everything updates in real time; the operator never refreshes.

9. How It Is Built and Where It Runs

The whole system is TypeScript, front to back. That is deliberate: the shape of an incident or an alert is declared once and used by both the browser and the server, so a change to the data model becomes a compile error rather than a bug discovered in testing — which matters in a system that issues public warnings.

Layer	Choice	Why
Frontend	React + TypeScript, Vite, Tailwind, Leaflet maps, PWA	Suits many screens; the PWA gives installability, push notifications and offline capability without building two native apps.
Backend	Node.js + TypeScript (Express or NestJS)	Same language as the frontend; the workload is almost entirely input/output, which suits Node's async model.
Database	PostgreSQL with PostGIS	Not negotiable. "Who is inside this polygon?" is the most frequent operation in the system, and PostGIS answers it with a spatial index in milliseconds.
Offline	Service Worker + IndexedDB + Background Sync	Queues emergency reports locally and sends them by itself when signal returns.

Every data source is free, and most need no API key at all:

Source	Provides	Key
USGS	Global earthquakes, updated every minute	None
Open-Meteo	Rainfall, wind, temperature and forecasts, worldwide	None
GDACS	Global multi-hazard alerts with severity already assigned	None
ReliefWeb (UN)	Declared ongoing disasters and situation reports	None
NASA FIRMS	Satellite-detected active fires	Free key
OpenStreetMap	Base maps, hospitals, shelters, road network	None
Browser geolocation	The user's live coordinates	Built in

Alerts go out over Web Push, a Telegram bot and email — all free. SMS has no free option anywhere, so it is built as a simulated gateway sitting behind a swappable adapter: the architecture is complete and provider-ready, and switching to a real provider is one class. This limitation is stated openly rather than hidden.

It runs on four free-tier platforms: **GitHub** for source, CI and the scheduled jobs that trigger data ingestion; **Vercel** for the React app; **Railway or Render** for the Node API and workers; and **Supabase** for PostgreSQL with PostGIS, authentication, row-level security, file storage and real-time streaming. **Total recurring cost: zero.**

10. What Is Deliberately Left Out

Scoping a project is as much about what you refuse to build as what you build. Four things were considered and consciously excluded:

Machine learning. It would have been easy to bolt a model onto the severity scoring and call the project "intelligent". It is deliberately rule-based instead. The rules are a published, readable table — rainfall above this figure means flood warning, this many corroborating reports raises confidence to that level. When the output of a system decides whether thousands of people are told to evacuate, being able to explain exactly why it produced a warning is more valuable than a slightly better score from a model nobody can interrogate. It is also far easier to defend under questioning than a black box.

Native Android and iOS apps. A progressive web app gives installability, push notifications and full offline capability from one codebase. Building and maintaining two native applications would consume weeks and add nothing this project needs.

A paid SMS gateway. There is no free option anywhere. Rather than pretend otherwise, SMS is implemented as a simulated gateway behind a swappable adapter, with an outbox screen showing exactly what would have been sent, to whom and when. The architecture is complete; only the paid provider is absent.

Satellite damage assessment from imagery. The relevant datasets run to tens of gigabytes and the compute required does not fit the timeline. It would consume the whole project for one feature.

11. Problems Identified Before Writing Any Code

Fifteen design problems were worked through on paper first. The six that shaped the architecture most:

Problem	How it is handled
False alarms would destroy public trust, but verifying everything by hand costs critical minutes	Four severity tiers. Official sources above threshold are trusted automatically because the issuing agency already verified them. Citizen-originated incidents need either corroboration from several independent reports or a named human verification. No mass alert ever goes out without one of the two, and the decision is logged.
One event produces hundreds of duplicate reports	Every incoming signal is checked against open incidents of the same hazard type within a configurable radius and time window. If one matches, the signal attaches to it and raises the confidence; if not, a new incident is created. The radius differs by hazard - an earthquake correlates over a much wider area than a collapsed wall.
Networks fail during the disaster itself	The app is cached on the device and opens with no connection. Reports go into local storage and are transmitted automatically by the browser when signal returns. Each report carries a client-generated ID so retrying can never create duplicates.
"Who is in this zone?" is slow across a large user table	Locations are stored as PostGIS geography points with a spatial index, and the query uses an index-accelerated proximity test rather than computing distance for every row. The before-and-after timing will be measured and reported.
Alert fatigue	Precise geofencing instead of district-wide broadcasts, four tiers so routine information never arrives with emergency urgency, per-user alert radius and quiet hours, and rate limiting so the same incident does not alert someone repeatedly.
No disaster will happen during evaluation	A scenario simulator injects a scripted disaster and drives the entire pipeline - ingestion, correlation, verification, geofenced alert, delivery tracking, team assignment, routing, resolution, analytics - in about ninety seconds, repeatably. It doubles as an integration test.

The other nine cover location privacy and consent, role and permission complexity, the absence of a public API from national weather authorities, paid social-media APIs, free-tier hosting limits, real-time delivery at scale, accessibility and language, the cost of SMS, and keeping an accountability record that cannot be tampered with.

12. Building It, Phase by Phase

The build is sequenced so that each phase ends in something that can be shown working, rather than leaving everything to prove itself at the end. The riskiest parts — geospatial querying and real-time delivery — are deliberately built early rather than last.

Phase	What gets built	What proves it worked
Foundation	All four hosting platforms wired together, database schema with geospatial support, login with four roles, app shell and navigation	A logged-in user sees an empty map, deployed on a public URL
Ingestion	Adapters for four public data sources, duplicate rejection, scheduled polling, feed-health screen	Real earthquakes and live weather appear on the map by themselves
Intelligence	Correlation rules, severity scoring, incident lifecycle, verification workflow, live coordinator dashboard	One incident, many reports attached to it, updating without refresh
Alerting	Geofence matching with a spatial index, alert composer with map drawing, dispatch queue, three live channels, delivery tracker	Drawing a zone alerts only the people inside it, and delivery is counted
Citizen app	SOS flow, report form, offline queue, background sync, push notifications, safe check-in, shelter finder	Aeroplane mode on, submit a report, aeroplane mode off, it arrives
Coordination	Responder screens, assignment lifecycle, routing that avoids blocked roads, teams, resources, shelters	A full dispatch cycle from assignment to resolution
Admin and proof	Analytics, audit log, editable thresholds, scenario simulator, load testing	The simulator drives the entire pipeline end to end; every metric recorded

13. How Success Will Be Measured

The project will be judged on numbers rather than screenshots. Each of these will be measured and reported:

Measurement	Target
Time to find everyone inside an alert zone, across 100,000 users	under 500 ms
Time from a coordinator verifying to the alert reaching a device	under 30 seconds
Share of alerts successfully delivered	above 95 percent
Dashboard update delay after a change	under 2 seconds
Offline sync: 50 queued reports after connectivity returns	100 percent, no duplicates
Correctly grouping 200 synthetic reports of 5 known events	90 percent or better
Concurrent users under load test	500, with stable response times
Time for a citizen to submit an emergency report	under 10 seconds, 3 taps

14. In Short

Five steps describe the whole thing:

1. **Watch.** Public hazard feeds and citizen reports flow in continuously and automatically, around the clock.
2. **Decide.** Related signals merge into a single incident with a transparent, explainable severity score — deliberately rule-based, not a black box, because this decides whether people get told to evacuate.
3. **Verify.** A human confirms before the public is alerted, so false alarms never go out and public trust survives.
4. **Warn.** Only the people physically inside the danger zone are notified, across several channels, with delivery confirmed per person.
5. **Coordinate.** Teams are assigned, routed around blocked roads, tracked to resolution, and every action is written to a log that cannot be altered.

Monitoring on one side, communication on the other — exactly what the title describes. Detection is already solved by satellites, seismic networks and weather models. The last mile is not. The last mile is software, and that is where this project intervenes.